

Language as Graph Traversal: The DMLG Architecture

A dynamic, modular, explainable graph-based governance architecture for language

Joachim De Witte

May 2026

©2026 Joachim De Witte. All rights reserved.

Published under the Creative Commons BY-NC-ND 4.0 license.

Abstract

The Dynamic Modular Language Graph (DMLG) is introduced as a general-purpose, graph-based architecture for representing language as a deterministic, canonical structure rather than a statistical distribution. In contrast to transformer-based language models, which rely on high-dimensional embeddings and large-scale parameter scaling, DMLG models language as an explicit graph of tokens and transitions, enabling transparent structure, modularity, and fine-grained control.

By treating language as graph traversal, DMLG provides a unified substrate for multiple applications, with generation emerging only as a special case of traversal. Its primary contribution is a structural governance layer that can constrain, audit, filter, or guide any generative model—independent of architecture, scale, or training regime. This enables domain-locked behaviour, canonical style enforcement, explainable transitions, and deterministic safety guarantees that are difficult or impossible to obtain from purely statistical systems.

DMLG builds on earlier modular and paging-based architectures but extends them into a fully structural representation of linguistic space. The result is a lightweight, trainable, interpretable, and model-agnostic governance architecture that integrates seamlessly with existing LLMs and hybrid intelligence systems. By providing explicit structure and controllable transitions, DMLG addresses fundamental limitations of parameter-scaling approaches and offers a path toward safer, more transparent, and more modular AI systems.

Contents

1	Introduction	3
2	Motivation	3
3	Related Work	4
3.1	Language Models and Structural Alternatives	4
3.2	Governance, Safety, and Explainability	4
3.3	Graph-Based Reasoning and Traversal Models	4
3.4	Relation to Earlier Works by the Author	4
4	Definition	5
4.1	Language Graph: The Structural Substrate	5
4.2	Dynamic Layer: Context-Sensitive Navigation	5
4.3	Paging: Modular Biomes	5

4.4	Not a Language Model	6
4.5	A Linguistic Ontology	6
4.6	Cognitive Plausibility	6
4.7	Sentence-Level PRE	6
4.8	Summary	7
5	Architecture	7
5.1	Canonical Graph and Deterministic Tokens	7
5.2	Context Compression via Grammar	7
5.3	Pagination and State Management	8
5.4	Transition Evaluators and Custom Activations	8
5.5	Dynamic Vocabulary and Scaling Behaviour	8
5.6	Keywords, Beam Search, and Additional Steering	9
5.7	Sentence-Level PRE: Propose–Retry–Evaluate	9
5.8	Token-Level Rule-Based Coherence	10
5.9	Multi-Stage Governance Pipeline	10
6	Experimental Setup	10
6.1	Base Model	10
6.2	DMLG Training	10
6.3	Hybrid Sampling Procedure	11
6.4	Evaluation Tasks	11
6.5	Context Requirements	11
6.6	Beam-Search-Based Transition Expansion	12
6.7	Success Criteria	12
7	Structural Evaluation Mode	12
7.1	Confidence Scoring	12
7.2	Threshold-Based Governance	13
7.3	Advantages	13
7.4	Use Case: Canonical Style Monitoring	13
8	Limitations and Open Questions	13
8.1	Limitations	14
8.2	Open Questions	14
9	Testable Predictions	15
9.1	Prediction 1: Deterministic Structure Improves Governance	15
9.2	Prediction 2: Small, Interpretable Evaluators Are Sufficient	15
9.3	Prediction 3: Graph Quality Outperforms Parameter Scaling	15
9.4	Prediction 4: Hybrid Systems Outperform Pure LLMs	16
9.5	Prediction 5: General-Purpose Applicability Beyond Language	16
9.6	Prediction 6: Practical Feasibility With Small Teams	16
10	Conclusion	16
A	Triadic Cosmos Ecosystem Context	17

1 Introduction

Contemporary language models are built on large-scale statistical learning, high-dimensional embeddings, and transformer-based attention mechanisms. These systems have achieved remarkable generative capabilities, yet they lack explicit structure, deterministic control, and transparent internal representations. As a result, they remain difficult to govern, hard to interpret, and fundamentally limited in their ability to provide guarantees about domain fidelity, safety, or stylistic consistency. Parameter scaling has improved performance but has not resolved these structural limitations.

This paper introduces the Dynamic Modular Language Graph (DMLG), a general-purpose architecture that models language as a deterministic, canonical graph rather than a probability distribution. In DMLG, tokens and their allowable transitions form an explicit structural substrate, and language use is expressed as traversal through this graph. This shift from statistical modelling to structural representation enables transparency, modularity, and fine-grained control that are difficult to obtain from purely neural approaches.

The primary contribution of DMLG is not a new generative model, but a new *governance architecture*. By providing an explicit graph of canonical transitions, DMLG can constrain, filter, audit, or guide any generative system—independent of its internal mechanisms. This enables domain-locked behaviour, explainable transitions, deterministic safety constraints, and canonical style enforcement. Generation is treated only as a special case of graph traversal, rather than the central purpose of the architecture.

DMLG is lightweight, modular, trainable on small datasets, and compatible with existing large language models. It integrates naturally with hybrid intelligence systems, where a generative model provides expressive power and DMLG provides structure, control, and interpretability. By introducing an explicit structural layer, DMLG addresses fundamental limitations of parameter scaling and offers a path toward safer, more transparent, and more modular AI systems.

The remainder of this paper develops the motivation, related work, architectural details, and implications of modelling language as graph traversal, and positions DMLG as a structural foundation for future governance layers in intelligent systems.

2 Motivation

Large language models have demonstrated impressive generative capabilities, yet they remain fundamentally limited in their ability to provide guarantees about correctness, safety, domain fidelity, or stylistic consistency. Their internal representations are opaque, high-dimensional, and statistically entangled, making it difficult to audit behaviour, enforce constraints, or ensure predictable operation in sensitive or domain-specific settings. As these models are increasingly deployed in real-world applications, the need for governance, transparency, and controllability has become critical.

DMLG is motivated by the observation that many of the structural limitations of LLMs arise not from insufficient scale, but from the lack of an explicit, deterministic representation of linguistic structure. Rather than attempting to solve governance and safety through additional parameter scaling, DMLG introduces a complementary structural layer that can constrain, filter, or guide any generative model. This hybrid intelligence approach allows a statistical model to provide expressive power while a deterministic graph provides structure, control, and explainability.

Although this paper focuses on language, the underlying architectural principles are general. A deterministic, modular, dynamically learnable graph can serve as a governance substrate for any domain that can be expressed in terms of states and transitions. The resulting architecture is traceable, transparent, dataset-driven, canon-faithful, customizable, lightweight, computa-

tionally inexpensive, cloud-deployable, and usable on demand. These properties make DMLG suitable not only for language governance but for a broad class of structured reasoning and control tasks across domains.

Moreover, the architecture is practically realizable. Because DMLG relies on explicit structure and small, interpretable components rather than large-scale statistical training, it can be developed and deployed by a focused team of AI, language, and domain experts. With appropriate engineering effort, such a system could be brought into production within a relatively short time frame, providing a viable path toward safer, more transparent, and more controllable intelligent systems.

3 Related Work

3.1 Language Models and Structural Alternatives

Transformer-based language models Vaswani et al. 2017; Brown et al. 2020; Touvron, Lavril, Izacard, et al. 2023 form the dominant paradigm in contemporary NLP. These systems rely on high-dimensional embeddings, attention mechanisms, and large-scale statistical training. While highly capable, they provide limited transparency, weak controllability, and no explicit structural representation of language.

Several strands of research explore alternatives to purely statistical sequence modelling. Graph-based or symbolic approaches to linguistic structure Gildea 2002; Jurafsky and Martin 2023 aim to make syntax or semantics explicit, while neuro-symbolic hybrids Besold et al. 2017 and modular neural architectures Andreas et al. 2016 attempt to combine learned components with structured reasoning. However, these systems typically do not provide a deterministic, canonical token-level graph of language.

3.2 Governance, Safety, and Explainability

The need for controllable, auditable, and safe language systems has led to work on constrained decoding Hokamp and Liu 2017, rule-based filters, and post-hoc safety layers Ganguli, Askell, Bai, et al. 2022. Explainable AI frameworks Arrieta et al. 2020 and formal verification techniques Huang et al. 2017 offer partial solutions but do not yield a unified structural model that can act as a governance layer for generative systems. Existing approaches operate *outside* the model’s internal structure and therefore cannot provide deterministic guarantees about allowed transitions or domain boundaries.

3.3 Graph-Based Reasoning and Traversal Models

Graph traversal is widely used in semantic parsing, knowledge graph reasoning, and planning Hamilton, Ying, and Leskovec 2017; Kipf and Welling 2016, but these methods operate on external knowledge structures rather than modelling language itself as a canonical graph. Prior work on dynamic or modular graph architectures remains limited and is generally not applied to token-level linguistic structure or governance.

3.4 Relation to Earlier Works by the Author

This paper builds on several earlier works by the author.

First, *A Safe and Modular AGI Architecture* introduces the Paged-PRE-CAS framework for modular reasoning, paging-based context management, and evaluator-governed execution. The present work adopts the same design philosophy of explicit modular structure and deterministic control, but applies it specifically to language as a canonical graph.

Second, *A Dynamic and Modular Architecture for Language Modeling* presents DMLM, a generative architecture demonstrating that coherent language can emerge from modular, non-transformer components. In contrast, the current paper does not focus on generation: the Dynamic Modular Language Graph (DMLG) is introduced as a structural, deterministic representation of language, with generation treated only as a special case of graph traversal.

Third, *Adaptive Neural Networks* highlights the importance of custom activation functions and page-specific dynamics in modular systems. These ideas inform the design of DMLG’s local transition evaluators, which rely on small, interpretable components rather than large monolithic networks.

Finally, *Structural Limits for AI Scaling* argues that parameter scaling alone cannot yield systems with the structural guarantees, controllability, or explainability required for AGI-aligned behaviour.

The present work addresses this gap by proposing a graph-based governance architecture that provides explicit structure, deterministic transitions, and domain-level control, independent of the underlying generative model.

4 Definition

A Dynamic Modular Language Graph (DMLG) is a structural, deterministic, and context-sensitive language generator built on top of a canonical language graph. It extends the formal graph-theoretic foundations of language with dynamic traversal mechanisms, a non-statistical generative process, an explicit linguistic ontology, and cognitively plausible functional organisation. This section provides a high-level definition of the DMLG concept.

4.1 Language Graph: The Structural Substrate

A DMLG is grounded in a canonical language graph $G = (V, E)$, where:

- V is the set of canonical tokens (lemmas, grammar markers, structural symbols),
- $E \subseteq V \times V$ is the set of admissible transitions.

Edges are either deterministic (structural invariants) or functional (context-sensitive transitions scored by local evaluators). The graph is canon-closed: it cannot generate unseen tokens or transitions. This provides strong structural guarantees and forms the static backbone of the system.

4.2 Dynamic Layer: Context-Sensitive Navigation

A DMLG becomes dynamic by equipping the graph with:

- **local micro-models** (deterministic functions or small MLPs) that score transitions,
- **grammar-driven context compression** that activates relevant regions of the graph,
- **probabilistic traversal** that selects among admissible transitions.

Generation is not next-token prediction but navigation: a probabilistic walk through a deterministic structure guided by compressed context.

4.3 Paging: Modular Biomes

The node set is partitioned into pages—modular subgraphs representing semantic or structural biomes. Paging restricts traversal, reduces search complexity, and enables modular specialisation. Pages are activated or deactivated based on context, providing a structured mechanism for domain control and stylistic fidelity.

4.4 Not a Language Model

A DMLG is not a language model in the classical sense. It has:

- no embeddings,
- no attention,
- no autoregressive hidden state,
- no probabilistic next-token prediction.

It generates sentences through structural composition, modular micro-models, and context-sensitive traversal. It is therefore a *language generator*, not a language model, and occupies an architectural category orthogonal to transformers and recurrent networks.

4.5 A Linguistic Ontology

By making tokens, edges, and pages explicit, a DMLG exposes the ontological structure presupposed by language:

- tokens as ontological units,
- edges as grammatical or semantic relations,
- pages as modular regions,
- traversal as dynamic instantiation of meaning.

This yields a mechanistic ontology of language that is explicit, interpretable, and corpus-grounded.

4.6 Cognitive Plausibility

Although not designed as a cognitive model, the DMLG aligns with key principles of human language cognition:

- discrete lemmas and grammatical markers,
- modular semantic regions,
- local decision-making,
- constraint-based incremental processing,
- structural self-monitoring via PRE.

These parallels arise naturally from the architecture’s structural commitments.

4.7 Sentence-Level PRE

The Propose–Retry–Evaluate (PRE) mechanism provides sentence-level governance. PRE validates complete sentences using minimal deterministic checks (grammar validity, structural closure, biome consistency) and triggers retries when necessary. PRE complements token-level coherence rules and ensures global structural integrity.

4.8 Summary

A DMLG is defined by:

- a deterministic canonical graph,
- modular paging biomes,
- grammar-driven context compression,
- local micro-models for functional variation,
- probabilistic traversal through admissible transitions,
- and PRE-based structural governance.

It is a structural, interpretable, and cognitively plausible language generator whose power derives from explicit structure rather than statistical scale.

5 Architecture

The Dynamic Modular Language Graph (DMLG) refines earlier modular language architectures by making the structural representation of language fully explicit and deterministic. Rather than modelling language as a latent distribution over embeddings, DMLG represents it as a graph of canonical tokens and transitions, equipped with local evaluators that govern traversal. This section provides a high-level architectural overview, focusing on the aspects most relevant for governance and hybrid integration with large language models.

5.1 Canonical Graph and Deterministic Tokens

At the core of DMLG is a canonical graph whose nodes represent tokens in a normalized form (e.g., lemmas, grammar markers, or domain-specific symbols). Edges encode allowable transitions between these tokens, forming a deterministic structural substrate. Tokens are not tied to any particular surface form or language; the only requirement is that a canonical representation can be defined. This makes DMLG inherently language-agnostic and suitable for domains beyond natural language.

Deterministic tokens ensure that each graph state has a well-defined interpretation. This contrasts with embedding-based models, where meaning is distributed and context-dependent. In DMLG, the meaning of a state is given by its position in the graph and the transitions that lead to and from it.

5.2 Context Compression via Grammar

DMLG uses grammar as a deterministic mechanism for context compression. Instead of storing long surface sequences, the graph encodes structural patterns (e.g., phrase types, clause boundaries, discourse roles) as grammar tokens and transitions. Traversal through these grammar structures provides a compressed representation of context that plays a role analogous to attention, but in a fully explicit and deterministic form.

This grammar-based context acts as the primary signal for determining which transitions are permissible or preferred at each step. It allows DMLG to maintain long-range structural coherence without relying on high-dimensional continuous states.

5.3 Pagination and State Management

The graph is organized into pages or regions that group related tokens and transitions. Pagination provides a structured way to manage the in-out space of the graph: entering a page corresponds to focusing on a particular subdomain, style, or structural regime, while exiting a page corresponds to shifting context. Although the detailed paging mechanisms are left to future work, the architecture assumes that:

- pages can be activated or deactivated based on context,
- transitions across pages are explicitly controlled,
- evaluators can be specialized per page.

This organization supports modularity, interpretability, and efficient computation.

5.4 Transition Evaluators and Custom Activations

Transitions in DMLG are governed by local evaluators. These can be:

- purely deterministic (rule-based) functions,
- small MLPs with custom activation functions,
- page-specific evaluators specialized for a subset of the graph,
- global evaluators with dataset-specific activation laws.

Deterministic transitions define the structural backbone; learned evaluators modulate preference, not validity.

Custom activation functions allow the evaluators to reflect dataset-level properties, such as a “temperature law” or “scaling law” for a given book or corpus. In this view, the graph structure depends on both vocabulary size (tokens as nodes) and transition complexity (grammar + lemma combinations), while the evaluators encode how likely or preferred certain transitions are within that structure.

5.5 Dynamic Vocabulary and Scaling Behaviour

DMLG supports a dynamic vocabulary: tokens can be added, refined, or reorganized as the canonical graph evolves. This enables:

- adaptation to new domains or styles,
- incremental refinement of canonical forms,
- scaling from small, tightly curated graphs to larger, more expressive ones.

The architecture predicts that governance performance depends more on graph quality (coverage, structure, transition design) than on parameter count. This suggests a form of structural scaling law: improvements in canonical graph design yield larger gains than additional parameters in the evaluators.

5.6 Keywords, Beam Search, and Additional Steering

In addition to context and grammar-based transitions, DMLG can use keywords or other high-level signals as auxiliary steering inputs. During traversal, a bounded-depth, wide-beam search can explore multiple grammar–lemma transition sequences that correspond to candidate tokens or phrases. Evaluators then score these paths based on:

- compatibility with the current context,
- alignment with keywords or goals,
- canonical and grammatical validity.

This mechanism supports flexible yet controlled exploration of the graph, and integrates naturally with hybrid setups where DMLG filters or evaluates LLM outputs. Context remains the primary driver of transitions via the mini-MLP evaluators, while beam search and keywords provide additional, interpretable steering.

Overall, DMLG combines canonical graph structure, grammar-based context compression, dynamic vocabulary, and modular evaluators into a lightweight, deterministic architecture. This makes it suitable as a general-purpose governance layer that can be trained on specific datasets and applied across domains, while remaining compatible with existing generative models.

5.7 Sentence-Level PRE: Propose–Retry–Evaluate

Beyond token-level evaluators, DMLG incorporates a sentence-level validation layer based on the PRE (Propose–Retry–Evaluate) mechanism. This layer operates after a full sentence candidate has been generated (either by the LLM, by DMLG traversal, or by a hybrid sampler) and determines whether the sentence satisfies a minimal set of structural and semantic constraints. PRE provides a deterministic acceptance filter that complements token-level coherence checks.

Propose. A sentence candidate is proposed by the generative component. The sentence is represented in canonical form, consisting of grammar tokens, lemma tokens, and structural markers. No assumptions are made about the origin of the proposal: it may come from an LLM, from DMLG traversal, or from a hybrid system.

Evaluate. The proposed sentence is evaluated using a minimal set of deterministic checks, like grammar validity rules or trivial semantical checks such as repeated lemmas or degenerate loops.

These checks are intentionally minimal: they ensure structural validity without imposing stylistic or domain-specific constraints, which are handled by the graph and evaluators.

Retry. If the sentence fails evaluation, the system triggers a retry. A new proposal is generated, optionally using:

- a different sampling path,
- a fallback agent or page,
- a modified context state,
- or a constrained transition subset.

Retry is always sentence-based, never token-based. This ensures that token-level coherence and sentence-level validity remain conceptually separate.

5.8 Token-Level Rule-Based Coherence

PRE operates alongside a token-level coherence mechanism derived from the canonical grammar. This mechanism enforces local structural constraints during generation:

- punctuation may only follow specific grammar tokens,
- repeated tokens or near-repetitions are suppressed,
- conjugations must follow valid grammatical openings,
- transitions that violate canonical adjacency rules are rejected.

These rule-based checks act as a lightweight, deterministic coherence filter that prevents invalid transitions before they accumulate into sentence-level errors.

5.9 Multi-Stage Governance Pipeline

Together, token-level coherence and sentence-level PRE form a multi-stage governance pipeline:

1. **Token stage:** rule-based grammar coherence filters invalid transitions.
2. **Sentence stage:** PRE validates full sentences and triggers retries when necessary.
3. **Graph stage:** evaluators score transitions and maintain canonical consistency.

This layered design ensures that DMLG maintains structural integrity at both local and global levels, providing deterministic governance that remains lightweight, interpretable, and compatible with hybrid LLM+DMLG systems.

6 Experimental Setup

To evaluate the Dynamic Modular Language Graph (DMLG) as a governance layer for generative models, we propose an experimental setup in which a large language model (LLM) is guided or constrained by a DMLG trained on a specific dataset. The goal is to assess whether hybrid systems (LLM + DMLG) exhibit improved safety, consistency, and canonical faithfulness without degrading the underlying model’s generative capabilities.

6.1 Base Model

Any open-weights LLM with a standard token-level sampling interface may be used; for concreteness, a model such as *gpt-oss-120b* serves as a representative example. The LLM is left unmodified: no fine-tuning, no architectural changes, and no additional training. The only intervention occurs at sampling time, where DMLG influences token selection.

6.2 DMLG Training

A DMLG is trained on a curated dataset representing a specific domain, style, or canonical form. The graph encodes:

- a canonical vocabulary (tokens or lemmas),
- allowable transitions between tokens,
- local evaluators that score or filter transitions,

- optional domain-specific constraints.

Although DMLG typically uses lemma-based canonical forms, the architecture is language-agnostic. The only requirement is that a canonical representation can be constructed; tokens need not be English lemmas. For ease of integration, the DMLG may dynamically align its canonical vocabulary with the LLM’s tokenization, enabling seamless hybrid sampling.

6.3 Hybrid Sampling Procedure

During generation, the LLM produces a probability distribution over its vocabulary at each step. Instead of sampling directly from this distribution, the DMLG:

1. receives the current graph state and the LLM’s logits,
2. filters or reweights tokens based on canonical transitions,
3. selects the next token consistent with the DMLG structure,
4. updates its internal state and continues traversal.

This setup allows the DMLG to enforce domain fidelity, stylistic consistency, and safety constraints while preserving the expressive power of the LLM.

6.4 Evaluation Tasks

To test governance performance, the hybrid system is evaluated on tasks related to the dataset used to train the DMLG. Representative tasks include:

- **Canonical style reproduction:** generating text in the style of a book or corpus without copying or memorization.
- **Domain-locked reasoning:** answering questions or producing explanations constrained to the domain.
- **Safety and consistency tests:** adversarial prompting to assess whether the DMLG prevents off-domain or unsafe outputs.
- **Stability under sampling variation:** measuring robustness across temperatures and decoding strategies.

A particularly informative experiment is to train a DMLG on a single book with a distinctive style and evaluate whether the hybrid system can produce canon-faithful text that respects the book’s structure and tone without reproducing copyrighted content. This tests whether DMLG can guide an LLM toward structured, style-consistent behaviour while avoiding memorization.

6.5 Context Requirements

The DMLG maintains its own compact context state, which is typically far smaller than the LLM’s context window. This enables efficient traversal and governance even for long sequences. The LLM’s context is used normally; the DMLG’s context acts as an additional structural signal rather than a replacement.

6.6 Beam-Search-Based Transition Expansion

In many cases, a single LLM token does not correspond to a single canonical transition in the DMLG. Instead, producing a token typically requires a short sequence of transitions, such as a grammar token followed by a lemma token. Thus, for each candidate LLM token, the DMLG must consider multiple possible transition paths rather than a single edge.

To handle this, we introduce a beam-search expansion mechanism. For each LLM token under consideration, the DMLG performs:

1. a bounded-depth beam search over all valid grammar–lemma transition sequences,
2. with a wide beam to capture all structurally permissible paths,
3. producing a set of candidate transitions corresponding to the same LLM token.

The resulting set of valid paths is then used to filter or reweight the LLM’s logits. Tokens for which no valid DMLG path exists are suppressed; tokens with multiple valid paths are scored according to evaluator weights and transition likelihoods within the graph.

This mechanism allows the DMLG to operate over the full vocabulary of the LLM, even when the canonical representation differs from the LLM’s tokenization. Although DMLG typically uses lemma-based canonical forms, the architecture is language-agnostic: any token can be represented as a sequence of grammar and non-grammar transitions, provided that a canonical form is available.

6.7 Success Criteria

The hybrid system is considered successful if:

- governance improves without collapsing the LLM’s generative diversity,
- outputs remain coherent and fluent,
- domain fidelity and canonical consistency increase,
- safety violations decrease under adversarial prompting,
- the system remains stable across long sequences.

This experimental setup provides a concrete, reproducible framework for evaluating DMLG as a structural governance layer and for assessing its ability to complement or correct the behaviour of large language models.

7 Structural Evaluation Mode

In addition to guiding or constraining token selection during sampling, the DMLG can operate in a non-intrusive evaluation mode. In this mode, the LLM generates text freely, and the DMLG assigns a confidence score to each token or sequence based on its canonical transition structure. This enables governance without modifying the sampling process.

7.1 Confidence Scoring

For each generated token, the DMLG evaluates whether the transition from the current graph state to the next is:

- structurally valid,

- canon-consistent,
- stylistically aligned with the training corpus,
- compliant with domain-specific constraints.

The DMLG produces a confidence score reflecting the degree to which the LLM output adheres to the canonical graph. A threshold mechanism can then determine whether the LLM remains within acceptable bounds.

7.2 Threshold-Based Governance

A simple threshold-based controller can be used to detect deviations:

- If the confidence score remains above the threshold, the LLM is considered to operate within canonical constraints.
- If the score falls below the threshold, the system may trigger a warning, request regeneration, or switch to a safer fallback mode.

This approach provides governance without interfering with the LLM’s sampling dynamics, making it robust and easy to integrate.

7.3 Advantages

Structural evaluation mode offers several benefits:

- **Non-intrusive:** the LLM remains unchanged and unmodified.
- **Model-agnostic:** works with any LLM that exposes token-level outputs.
- **Lightweight:** requires only graph traversal and evaluator scoring.
- **Explainable:** deviations can be traced to specific transitions.
- **Practical:** suitable for real-time monitoring and safety auditing.

7.4 Use Case: Canonical Style Monitoring

A practical experiment is to train a DMLG on a book or corpus with a distinctive style and use structural evaluation mode to assess whether an LLM’s generated text remains canon-faithful. Because the DMLG does not influence sampling, this setup isolates the evaluation capability and provides a clean test of structural consistency.

This mode demonstrates that DMLG can function not only as a governance layer during generation but also as an independent structural evaluator, broadening its applicability across domains and architectures.

8 Limitations and Open Questions

While the Dynamic Modular Language Graph (DMLG) provides a structural and deterministic foundation for governance, several limitations and open questions remain. These concern the construction of canonical graphs, the integration with generative models, the scalability of evaluator design, and the broader applicability of the architecture beyond language.

8.1 Limitations

Graph construction and coverage. DMLG relies on the availability of a high-quality canonical graph of tokens and transitions. Constructing such graphs requires domain expertise, careful curation, and iterative refinement. Incomplete or biased graphs may lead to overly restrictive behaviour or gaps in coverage, limiting the system’s expressiveness.

Evaluator design and training. Although DMLG uses small, interpretable evaluators, their design still requires empirical tuning and domain knowledge. The optimal balance between rule-based structure and learned evaluators remains an open engineering challenge, particularly for highly specialized domains.

Integration overhead with LLMs. Hybrid systems combining DMLG with large language models introduce additional orchestration complexity. Ensuring low-latency interaction, maintaining coherence across model boundaries, and preventing conflicts between statistical and structural components require careful system design.

Dependence on canonicalization quality. DMLG assumes that language can be represented through a canonical set of tokens and transitions. While effective for many domains, this assumption may be challenged by highly creative, ambiguous, or non-standard linguistic phenomena.

Limited empirical validation. Although the architecture is conceptually motivated and practically realizable, large-scale empirical validation remains future work. Systematic benchmarks comparing pure LLMs, hybrid systems, and structure-first approaches are needed to quantify the benefits and trade-offs.

8.2 Open Questions

How large should canonical graphs be? It remains unclear how graph size, granularity, and transition density affect governance performance. Determining optimal graph complexity for different domains is an open research direction.

What is the ideal division of labour between structure and scale? Hybrid systems raise fundamental questions about how much responsibility should lie with deterministic structure versus statistical models. The boundary between the two is not yet theoretically characterized.

Can canonical graphs be learned automatically? While DMLG currently assumes curated graphs, it is an open question whether high-quality canonical structures can be learned from data, constructed through self-supervision, or induced from model behaviour.

How far does general-purpose applicability extend? DMLG predicts that deterministic, modular graphs can govern any domain expressible as states and transitions. Testing this prediction across robotics, workflow control, simulation, and planning remains an important direction for future work.

What are the limits of interpretability? Although DMLG improves transparency, the interaction between evaluators, graph structure, and generative models may still produce emergent behaviour. Understanding the interpretability limits of hybrid systems is an open challenge.

Together, these limitations and open questions outline a research agenda for evaluating and extending DMLG as a structural, general-purpose governance architecture for intelligent systems.

9 Testable Predictions

The Dynamic Modular Language Graph (DMLG) makes several concrete and falsifiable predictions about the behaviour of hybrid intelligence systems, the structure of linguistic space, and the limits of statistical language models. These predictions can be evaluated through empirical experiments, ablation studies, and controlled integration with existing LLMs.

9.1 Prediction 1: Deterministic Structure Improves Governance

DMLG predicts that introducing an explicit, deterministic graph of canonical transitions will measurably improve governance performance in hybrid systems. When an LLM is constrained or guided by DMLG, we expect:

- a reduction in off-domain or stylistically inconsistent outputs,
- improved adherence to domain-specific rules and canonical forms,
- fewer safety violations under adversarial prompting,
- more stable behaviour across temperature and sampling settings.

These effects should be observable even when the underlying LLM remains unchanged, demonstrating that governance can be improved structurally rather than statistically.

9.2 Prediction 2: Small, Interpretable Evaluators Are Sufficient

DMLG predicts that local transition evaluators—implemented as small, interpretable components—are sufficient to maintain coherence, structure, and domain fidelity. In particular:

- evaluator modules with orders of magnitude fewer parameters than an LLM can reliably enforce canonical transitions,
- evaluator performance scales with graph quality rather than parameter count,
- replacing evaluators with larger neural components yields diminishing returns.

This prediction can be tested by varying evaluator size while holding the graph structure constant.

9.3 Prediction 3: Graph Quality Outperforms Parameter Scaling

DMLG predicts that improvements in the canonical graph—its coverage, structure, and transition quality—yield larger gains in governance accuracy than additional LLM parameter scaling. Formally:

- upgrading the graph while keeping the LLM fixed should improve constraint satisfaction and domain fidelity,
- scaling the LLM while keeping the graph fixed should yield smaller improvements,
- the two effects are not additive: structure dominates scale.

This prediction challenges the assumption that scaling alone is the primary path to improved reliability.

9.4 Prediction 4: Hybrid Systems Outperform Pure LLMs

DMLG predicts that hybrid systems—LLM + DMLG—will outperform pure LLMs on tasks requiring structure, consistency, or rule adherence. This includes:

- multi-step reasoning with explicit constraints,
- domain-locked generation (legal, medical, technical),
- style-preserving rewriting,
- safety-critical or compliance-sensitive tasks.

Performance should improve even when the LLM is weaker than state-of-the-art, demonstrating that structure can compensate for scale.

9.5 Prediction 5: General-Purpose Applicability Beyond Language

Although DMLG is instantiated here for language, the architecture predicts that deterministic, modular, dynamically learnable graphs can serve as governance substrates for any domain expressible as states and transitions. We expect:

- successful application to non-linguistic domains such as workflow control, robotics, planning, or simulation,
- similar gains in safety, consistency, and controllability,
- comparable performance using domain-specific graphs of similar size and complexity.

This prediction is falsifiable by implementing DMLG-style graphs in non-linguistic settings and measuring governance performance.

9.6 Prediction 6: Practical Feasibility With Small Teams

Finally, DMLG predicts that such systems are practically realizable. Because the architecture relies on explicit structure and small components rather than large-scale training, a focused team of AI, language, and domain experts should be able to construct a production system within a short development cycle. This can be tested by incremental prototypes and real-world deployments.

Together, these predictions provide a concrete empirical basis for evaluating DMLG as a structural, general-purpose governance architecture and for assessing its potential to complement or correct the behaviour of large language models.

10 Conclusion

This paper introduced the Dynamic Modular Language Graph (DMLG), a structural and deterministic architecture for representing language as a graph of canonical transitions rather than a statistical distribution. By shifting from implicit embeddings to explicit structure, DMLG provides a transparent, modular, and controllable substrate that can govern, constrain, or guide any generative model. Generation becomes a special case of graph traversal, while governance emerges as the primary function of the architecture.

DMLG addresses several fundamental limitations of contemporary language models. It offers deterministic control where statistical models provide only probabilistic tendencies, explicit structure where embeddings provide only latent geometry, and transparent transitions where

attention mechanisms provide only distributed influence. As a result, hybrid systems combining DMLG with existing LLMs can achieve improved safety, consistency, domain fidelity, and explainability without requiring additional parameter scaling.

Although instantiated here for language, the architectural principles underlying DMLG are general. Any domain expressible as states and transitions can, in principle, be governed by a deterministic, dynamically learnable graph. This positions DMLG as a candidate general-purpose governance layer for intelligent systems, extending beyond language to structured reasoning, workflow control, planning, and other domains where transparency and control are essential.

The work presented here outlines a structural foundation rather than a finished system. Future research must refine graph construction methods, evaluate hybrid performance at scale, and explore applications beyond language. Nonetheless, the architecture is practically realizable and offers a concrete path toward safer, more transparent, and more modular AI systems. By introducing explicit structure where current models rely on statistical inference alone, DMLG provides a new direction for the development of controllable and trustworthy intelligent systems.

A Triadic Cosmos Ecosystem Context

This paper is part of the *Triadic Cosmos* ecosystem, which develops a unified conceptual and computational framework spanning physical ontology and modular architectures for intelligent systems. All materials, extended notes, and evolving documentation are available in the public repository: <https://github.com/triadic-cosmos>. A complete reference Python implementation is included in the ecosystem under the AGPL-3.0 license, together with example datasets and models provided in a separate data repository. These resources make the architecture fully inspectable, reproducible, and practically deployable.

References

- Andreas, Jacob, Marcus Rohrbach, Trevor Darrell, and Dan Klein (2016). “Neural Module Networks”. In: *Proceedings of CVPR*.
- Arrieta, Alejandro Barredo et al. (2020). “Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI”. In: *Information Fusion* 58, pp. 82–115.
- Besold, Tarek R, Artur d’Avila Garcez, Sebastian Bader, Howard Bowman, Pedro Domingos, Pascal Hitzler, et al. (2017). “Neural-Symbolic Learning and Reasoning: A Survey and Interpretation”. In: *arXiv preprint arXiv:1711.03902*.
- Brown, Tom B, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. (2020). “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems*.
- Ganguli, Deep, Amanda Askell, Yuntao Bai, et al. (2022). “Red Teaming Language Models to Reduce Harms: Methods, Scaling Behaviors, and Lessons Learned”. In: *arXiv preprint arXiv:2209.07858*.
- Gildea, Daniel (2002). “Synchronous Tree-Adjoining Grammars for Machine Translation”. In: *Proceedings of ACL*.
- Hamilton, William L, Rex Ying, and Jure Leskovec (2017). “Inductive Representation Learning on Large Graphs”. In: *Advances in Neural Information Processing Systems*.
- Hokamp, Chris and Qun Liu (2017). “Lexically Constrained Decoding for Sequence Generation Using Grid Beam Search”. In: *Proceedings of ACL*.
- Huang, Ruitong, Bing Xu, Dale Schuurmans, and Csaba Szepesvári (2017). “SafetyNet: Detecting and Rejecting Adversarial Examples Robustly”. In: *Proceedings of ICCV*.
- Jurafsky, Daniel and James H Martin (2023). *Speech and Language Processing*. 3rd edition draft. Prentice Hall.
- Kipf, Thomas N and Max Welling (2016). “Semi-Supervised Classification with Graph Convolutional Networks”. In: *arXiv preprint arXiv:1609.02907*.
- Touvron, Hugo, Thibaut Lavril, Gautier Izacard, et al. (2023). “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv preprint arXiv:2302.13971*.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin (2017). “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*.